

Integrity and Signing in Streaming Measurement Data Formats

The OSF5 Approach: Corruption Detection, Cryptographic Proof of Origin, and PKI-Backed Third-Party Verifiability for Time-Series Measurement Data

Authors:

optiMEAS GmbH, Friedrichsdorf · optiMEAS Switzerland GmbH, Oberstammheim

Corresponding author:

Burkhard Schranz

Version:

1.1 · August 2026

License:

CC BY 4.0 (Creative Commons Attribution 4.0 International)

Changes from version 1.0 (June 2026): Sections 2 and 8 have been brought up to the August 2026 state — the CRC level is now a normative part of the specification and implemented in all five reference implementations. The technical content of sections 3 through 7 is unchanged. An English edition is available for the first time.

Abstract

Measurement data from industrial and railway applications increasingly underpins billing, warranty decisions, and regulatory evidence. At the same time, the EU Cyber Resilience Act (Regulation (EU) 2024/2847) obliges manufacturers of products with digital elements to protect data integrity. Established measurement data formats (MDF4, TDMS, HDF5, CSV) provide no built-in mechanism for this. This paper describes a three-level integrity profile for the Open Streaming Format version 5 (OSF5): a per-block CRC32C checksum for corruption detection; a SHA-256 hash chain with periodic Ed25519 signature anchors providing streaming-capable tamper evidence and proof of origin; and an X.509-based certificate model that lets uninvolved third parties verify a file offline, without recourse to the manufacturer. The design remains fully backward compatible, preserves the format's robustness against power loss, and is mapped against the threat catalog of DIN EN 50159. The publication also serves as a defensive publication: the methods described are meant to remain freely implementable.

1. Motivation

Streaming measurement data formats for embedded systems have solved the same core problems for years: lossless continuous recording, robustness against power loss, efficient block-wise processing. What they do not answer are two questions that regulation and the data economy are making urgent:

Is this file unchanged? — Bit errors on storage media, interrupted transfers, and faulty tool chains corrupt data unnoticed.

Does this file really come from this device? — As soon as measurement data moves money or liability (energy billing, warranty in rail vehicles, condition monitoring of critical infrastructure), asserting its origin is no longer enough; the origin must be provable — ideally to third parties who do not have to trust the operator.

Annex I of the EU Cyber Resilience Act requires the integrity of stored, transmitted, and processed data to be protected against unauthorized manipulation. For manufacturers of measurement equipment, a data format with integrity protection built in is therefore an immediate compliance building block.

A format comparison shows the gap: MDF4, TDMS, HDF5, and CSV have no built-in integrity check (HDF5 offers optional Fletcher32 as a filter, without proof of origin). Apache Parquet offers tamper evidence through AES-GCM-encrypted columns — coupled to symmetric encryption, so the statement "unchanged" holds only towards key holders and no third-party-verifiable origin arises. The approach described here closes precisely this gap, using asymmetric signatures and public certificate validation.

2. The Open Streaming Format in Brief

OSF is a binary, block-oriented format for time-series measurement and process data (optiMEAS GmbH; specification and implementations published openly at github.com/optimeas/osf). A file consists of an ASCII header line (OSF5 <metablock length>\n), a JSON metablock (channels, data types, metadata), and a stream of self-contained data blocks. Every block carries a channel index, a length field, and a control byte; the file is append-only and stays readable up to the last complete block after a hard abort. These properties — the self-contained nature of the blocks above all — are the foundation on which integrity can be added in a streaming-capable way.

The normative source for the base format is the published OSF specification (github.com/optimeas/osf, directory docs/; identical at docs.optimeas.com). The CRC level described in section 4 has been a normative part of it since the 2026-07-07 revision. The signature and PKI levels (sections 5 and 6) are described here in full for the first time and will feed into a coming revision of that same specification; those sections are therefore deliberately self-contained.

3. The Three-Level Integrity Profile

The profile is defined as a strictly ordered ladder; each level contains the one below it. The level applies per file and is declared exactly once.

Level	Protects against	Typical use
none	—	Laboratory, intermediate files, minimal embedded writers
CRC	Corruption (bit errors, truncation, media faults)	Standard for field data
Signed	Corruption and manipulation; origin provable to third parties	Operational data used as evidence

Two design rules prevent the format from fragmenting: writers choose the level freely (embedded systems keep the option of a minimal writer), but every conforming reader must handle all levels — the result is one format with a profile, not three dialects. And the granularity is always the whole file: either all blocks carry the mechanism or none do.

4. The CRC Level: Per-Block Corruption Detection

Declaration in the header. The header line is extended by an optional token:

```
OSF5 84512 crc32c:9A3F01BC\n
```

The token declares the algorithm for the entire file and at the same time carries the CRC32C checksum of the metablock — the only part of the file that cannot protect itself. The verification order is therefore deterministic: read the token, check the metablock CRC, parse the metablock, check the data blocks.

Fail-closed semantics. Header tokens are defined as "must understand": a reader that does not know a token rejects the file. This strictness is not a stylistic choice but a necessity — with variable-length data types (strings, binary data) the payload is defined solely by the length field; an integrity-unaware reader would hand appended checksum bytes to the application as payload and thereby produce silent data corruption. Fail-open is ruled out by construction.

Frame checksum per block. Every data block receives a CRC32C (Castagnoli; hardware-accelerated on x86/SSE4.2 and ARMv8) over the entire frame — channel index, length field, control byte, and payload. The scope deliberately includes the length field: a flipped length field is the most severe single error, because after it the reader loses the block boundaries. The checksum forms the last four bytes of the data area and is counted within the length field; the framing therefore stays intact for every reader, and the effective payload length is the block length minus four.

Structure of a data block without and with an active profile (LEN = value of the length field):

```
without: [channel index][length field][control byte][ payload ..... ]
                                                |<----- LEN ----->|
with:    [channel index][length field][control byte][ payload ..... ][CRC32C]
                                                |<----- LEN ----->|
                                                |<===== scope of the frame CRC =====>|
```

Error behavior. A CRC error in a data block invalidates exactly that block; the reader skips it, counts it in the read statistics, and continues — partial data is better than none. A metablock CRC error, by contrast, causes the file to be rejected, since nothing is interpretable without a trustworthy metablock.

5. The Signed Level: Hash Chain with Signature Anchors

Asymmetric rather than symmetric. An HMAC approach is ruled out: whoever can verify could also forge. The fitting model for device data is the digital signature — the private key stays in the device, verification uses the public one. The choice is Ed25519 (32-byte keys, 64-byte signatures, deterministic, RFC 8410) with SHA-256 as the chain hash. The CRC layer is fully preserved alongside it: it allows the fast per-block corruption check without cryptography, while the signature layer supplies provability.

A streaming-capable construction. A single signature at file close would be incompatible with the streaming principle (on power loss it would be missing entirely); a signature per block would be disproportionately expensive. The solution is a running hash chain with periodic anchors:

```
H(0) = SHA256(header line || metablock)
H(i) = SHA256(H(i-1) || frame_i)
```

At a configurable cadence (time- or block-based, plus on close) the writer emits a signature block — a new control byte type — carrying a consecutive anchor sequence number, the current chain hash, an Ed25519 signature over it, and a key or certificate reference. The first anchor thereby also covers the header and the metablock; signature blocks themselves carry a frame CRC like any other block.

Honest power-loss semantics. If recording aborts hard, everything up to the last anchor is signed; the tail after it is CRC-valid but unsigned. The verification report states exactly that ("signed up to timestamp X"). Beyond checking individual blocks, the chain also detects deletion, insertion, and resequencing of blocks; the anchor sequence numbers detect repetition within the file.

Transformations end the signature domain — by design. Merging, converting, and exporting produce files without a device signature as a matter of principle. Tools behave in a defined way here: the result falls back to the CRC level and records its provenance (source files together with their verification status) in the metadata. Optionally an organization can re-sign the result with a certificate of its own — that is a different statement from the device signature, and is marked as such.

6. PKI: Proof of Origin for Third Parties

Certificates rather than a key directory. So that not only the manufacturer but any third party can verify the origin, device keys are certified by a certificate authority (X.509 with Ed25519 per RFC 8410) — analogous to the trust model of HTTPS, but as a private, publicly documented hierarchy: a root CA (offline, hardware-protected), an issuing CA (connected to production or the device cloud), and beneath it device certificates whose subject carries the device serial number. Device certificates and organization or tool certificates (section 5) form separate, distinguishable classes.

The file carries its chain. The device certificate and the intermediate certificate are embedded once per file; the signature blocks reference them. The root certificate is never embedded — the trust anchor must, as a matter of principle, come from outside. A file is thereby a self-contained piece of evidence: verification works offline, without a directory lookup and without any involvement of the manufacturer; all that is needed is the publicly published root certificate (website and open repository, each with a fingerprint), which ships with the verification tools.

Validity at the time of signing. Measurement data outlives certificate lifetimes. The verification semantics are therefore: *was the certificate valid at the time of signing?* — a file recorded in 2027 still verifies positively in 2040. Combined with long device certificate lifetimes this is practical for measurement data; the theoretical residual weakness (backdating with a stolen, expired device key) is documented and can be closed with RFC 3161 timestamps if needed, without changing the format — the timestamp attaches to the signature, not to the data blocks. Revocation uses signed certificate revocation lists with best-effort semantics; the verification report states the status of the revocation check explicitly.

7. Relationship to DIN EN 50159

EN 50159 addresses safety-related communication in transmission systems for railway signaling and defines a catalog of basic threats. Although the profile described here is derived from the Cyber Resilience Act and addresses data at rest rather than transmission channels, the comparison is worthwhile — particularly for use in railway environments:

Threat (EN 50159)	Mechanism in the OSF5 profile
Corruption	Frame CRC32C per block; cryptographically: hash chain
Deletion	Hash chain (a missing block breaks the chain)
Insertion	Hash chain and signature (a foreign block breaks the chain)
Resequencing	Hash chain (order is part of forming the chain)
Masquerade	Ed25519 signature with device certificate and CA chain
Repetition	Anchor sequence numbers (within the file); file identity via a unique file UUID in the metablock as an anchor for system-level checking
Delay	Not applicable to data at rest

Two delimitations should be noted. First, the profile addresses repetition and deletion *at file level* (replaying old files, whole files disappearing) deliberately only through the file UUID interface: completeness across file boundaries is the task of the superordinate system level — the measurement environment, its sequence monitoring and secure time source — and is solved there. Second, strictly: the profile is a security mechanism in the sense of a category 3 environment (open networks) and makes no safety claim; it does not replace safety-related transmission per EN 50159 and does not establish SIL suitability.

8. Compatibility, Cost, Implementations

The profile is a revision of the OSF5 definition, not a new format: files without a token remain valid unchanged, and the predecessor version OSF4 is unaffected. The runtime overhead is small (CRC32C in the GB/s range, SHA-256 in the range of hundreds of MB/s, an Ed25519 signature in the microsecond range per anchor); the storage overhead is four bytes per block, roughly 100–120 bytes per signature anchor, and a one-off one to two kilobytes of certificate chain per file.

Open implementations of the format exist in Rust (core, with Python bindings), C++, Delphi, and Java under the MIT license; the format specification is published under CC BY 4.0. The integrity levels are being rolled out in stages: the CRC level is complete in all five implementations and is held in place by a shared set of reference files that the implementations' conformance tests check in common. Signing and PKI follow. Reference and negative test data (flipped bytes, broken chains, expired and revoked certificates) are part of the test suite; a strictly separated test PKI ensures that test certificates never produce productive verification results.

9. Conclusion

Streaming robustness and cryptographic integrity are not mutually exclusive — they reinforce one another when the mechanisms are designed along the block structure of the format: checksums per frame, chains across frames, signature anchors in the stream, certificates in the file. The contribution of this paper is the combination of these known building blocks into a graduated, backward-compatible integrity profile for streaming measurement data that third parties can verify offline. The methods are published for free implementation; the authors welcome feedback from the field, in particular from the railway domain.

License. This work is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Attribution: "optiMEAS GmbH and optiMEAS Switzerland GmbH".

Contact. optiMEAS GmbH, Friedrichsdorf · optiMEAS Switzerland GmbH, Oberstammheim · github.com/optimeas/osf · docs.optimeas.com